JAVA
TECH COMMUNITY

SERIES #1: CRACK MNC JAVA INTERVIEW

About Series #1: Crack MNC Java Interview

📘 **Who Is It For:**

- This series is designed for freshers, graduates, and professionals with 1-5 years of experience.
- It focuses on FAQ questions relevant to major MNC interviews.

🚀 **How to Use This Guide:**

- Use this to learn how real-time interviews ask questions 🎯.
- Focus on applying concepts and showcasing your project experience 💻🚀.

DAY #3: Java Abstraction and abstract class - Java FAQ 2025 🎯

🌟 What is Abstraction? 🌟

- 📌 Abstraction is one of the OOP principles.
- 💡 It is an OOP programming practice of hiding the implementation of an object and showing only the essential information to the end user.
- 🔒 We can achieve security by hiding the implementation of an object.
- 📊 99% of Java applications use Abstraction.
- 🚫 We won't show the implementation to the outside of the application.

DAY #3: Java Abstraction and abstract class - Java FAQ 2025 🎯

🌟 How to Achieve Abstraction? 🌟

- 🛠️ **We can achieve abstraction in 2 ways:**
- 🔷 Using **Abstract Class** (0% – 100%)
- 🔷 Using **Interface** (100%)

🌟 How to Achieve Abstraction Using an Interface? 🌟

1. 📄 **Create an Interface**
2. ✏️ **Declare all the required methods** in the interface body.
3. 🏗️ **Create an Implementation Class:**
   - 🛠️ The implementation class **must provide implementation** for all the methods declared in the interface.
4. 🖱️ **Access Methods:**
   - Use the implementation class object to access the methods.

DAY #3: Java Abstraction and abstract class - Java FAQ 2025 🎯

❓ Is Abstraction and Abstract Class the Same? ❓

- ❌ **No, both are different things.**

1. 🌟 **Abstraction**:
   - Abstraction is one of the **OOPS principles**.
   - It involves **hiding the implementation** of an object and showing only the **essential information** to the end user.
2. 📄 **Abstract Class**:
   - An **abstract class** is a type of class declared using the `abstract` keyword.
   - It is one of the **ways to implement the abstraction idea** in Java.

DAY #3: Java Abstraction and abstract class - Java FAQ 2025 🎯

📄 Abstract Class and Its Rules (Very Important for Interview) 📄

- 🔑 Basically, it is a **class**, and it is declared by using the **abstract** keyword.
- ✨ It can have both **abstract** and **non-abstract methods**.
- 🔄 **Abstract class methods** must be implemented in its **subclass**.
- 🏗️ It can have a **constructor** and **static methods**.
- 🚫 We can have a **final method**, but in the **subclass**, we can't change the final method implementation.
- ❌ We cannot create an object **directly**.
- 📌 If a **subclass** has some unimplemented methods, then the **subclass** must also be declared using the **abstract** keyword.
- 🖍️ Until we provide implementations for all the **abstract methods**.

DAY #3: Java Abstraction and abstract class - Java FAQ 2025 🎯

Sample Code of Abstract Class & Its Rules

```java
// 📄 Abstract Class and Its Rules (Very Important for Interview) 📄

// 🔑 Rule: Declaring an abstract class using the "abstract" keyword
public abstract class Vehicle {

    // ✨ Rule: Abstract classes can have both abstract and non-abstract methods
    protected String make;
    protected String model;
    protected int year;

    // 🏗️ Rule: Abstract classes can have constructors
    public Vehicle(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    // ✨ Abstract method (must be implemented in subclasses)
    public abstract void start();
    public abstract void stop();

    // ✨ Non-abstract method (can have implementation)
    public void displayInfo() {
        System.out.println("Make: " + make + ", Model: " + model + ", Year: " + year);
    }

    // 🚫 Rule: Abstract classes can have final methods, which cannot be overridden
    public final void printMessage() {
        System.out.println("This is a vehicle.");
    }

    // 🏗️ Rule: Abstract classes can have static methods
    public static void vehicleType() {
        System.out.println("General Vehicle Type");
    }
}
```

```java
// Subclass: Implementing abstract methods in the subclass
class Car extends Vehicle {

    public Car(String make, String model, int year) {
        super(make, model, year);
    }

    // 🔄 Rule: Abstract methods must be implemented in the subclass
    @Override
    public void start() {
        System.out.println("Car is starting...");
    }

    @Override
    public void stop() {
        System.out.println("Car is stopping...");
    }
}

// Subclass: Declared abstract because not all abstract methods are implemented
abstract class ElectricVehicle extends Vehicle {

    public ElectricVehicle(String make, String model, int year) {
        super(make, model, year);
    }

    // 🖍️ Rule: If a subclass doesn't implement all abstract methods, it must also be declared abstract
    public abstract void chargeBattery(); // Additional abstract method for Electric Vehicles
}

// Main Class to Test
public class Main {
    public static void main(String[] args) {
        // ❌ Rule: Cannot create an object of an abstract class
        // Vehicle vehicle = new Vehicle("Toyota", "Camry", 2020); // This will cause a compilation error

        // Valid: Creating an object of the subclass
        Car car = new Car("Toyota", "Camry", 2020);
        car.start(); // Abstract method implemented in Car
        car.displayInfo(); // Non-abstract method
        car.printMessage(); // Final method
        Vehicle.vehicleType(); // Static method
    }
}
```

Task for readers:

These are the most FAQ OOP basics! 🚀

- 💻 Explore More: Dive deeper into OOP FAQs, practice, and try small code snippets for each concept 💻. We'll share detailed code snippets on the community channel soon. For now, we want you to explore and learn! 🚀
- 🛠️ For Experienced Professionals: Reflect on how you've implemented these concepts in your current project.
- 🎓 For College Students & Graduates: Relate these concepts to your syllabus and try implementing small projects.

Pro Tip: Think Big, Start Small! 🌟 Let's grow together!

Note:

- This document is meant for brush-ups key concepts & Top FAQ.
- We always recommend working on projects and practicing hands-on coding to solidify your concepts. 💻✨